

Question Bank — Week 7

Queues (Lecture 07) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Answers are in a separate file: [week07-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. *SlowQueue* and *CircularQueue* are the classes of `dsa/queue.py`: `enqueue`, `dequeue` (raising `IndexError` when empty) and `Len`; *CircularQueue* also has `is_empty`, `is_full` and iteration front to back. A *CircularQueue* stores its items in a fixed Array with `head`, `size` and `capacity`, and the next `enqueue` writes to slot `(head + size) % capacity`. Stack is Week 6's.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W7-M01 – W7-M22
B	Short answer and essay	W7-E1 – W7-E5
C	Trace the code	W7-T1 – W7-T5
D	Ring state — draw every step	W7-S1 – W7-S3
E	Complexity analysis	W7-K1 – W7-K3
F	Find and fix the bug	W7-B1 – W7-B4
G	Write the code — checked by pytest	W7-C1 – W7-C5

Part A — Multiple choice

W7-M01 [what] "FIFO" means:

- **a)** the last item added is the first one removed
- **b)** the first item added is the first one removed
- **c)** items are removed in sorted order
- **d)** any item can be removed

W7-M02 [how] `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`, `enqueue(4)`, `dequeue()`.

What does the **next** `dequeue()` return?

- **a)** 4
- **b)** 1
- **c)** 2
- **d)** 3

W7-M03 [why] In `SlowQueue`, `dequeue` is `pop(0)` on a `DynamicArray`. Its cost on a queue of n items is:

- **a)** $O(n)$ — every remaining element shifts one slot left
- **b)** $O(1)$
- **c)** amortised $O(1)$
- **d)** $O(\log n)$

W7-M04 [why] An array-based queue that enqueues with `insert_at(0, x)` and dequeues with `pop()` at the end has costs:

- **a)** both $O(1)$
- **b)** enqueue $O(n)$, dequeue $O(1)$
- **c)** enqueue $O(1)$, dequeue $O(n)$
- **d)** both $O(n)$

W7-M05 [how] A `CircularQueue` with capacity 8 has `head = 6` and `size = 3`. The next enqueue writes to slot:

- **a)** 9
- **b)** 0
- **c)** 1
- **d)** 7

W7-M06 [how] A `CircularQueue` with capacity 5 has `head = 4` and `size = 2`. After one dequeue, `head` is:

- **a)** 5
- **b)** 3
- **c)** 1
- **d)** 0

W7-M07 [what] A ring buffer that stores only `head` and `tail` (no `size`) finds `head == tail`. The queue is:

- **a)** full
- **b)** empty
- **c)** either full or empty — it cannot tell which
- **d)** corrupted

W7-M08 [why] `CircularQueue` keeps a `_size` field because:

- **a)** it tells a full ring from an empty one, which look the same by `head` and `tail`
- **b)** `len` would otherwise be $O(n)$ with no other benefit
- **c)** the `Array` does not know its own length
- **d)** Python requires it for `__iter__`

W7-M09 [how] A full ring of capacity 4 holds `[E, F, C, D]` with `head = 2`. It grows to capacity 8 by copying `new[i] = old[i]` and keeping `head = 2`. Read front to back, the queue is now:

- **a)** C D E F
- **b)** C D None None

- **c)** E F C D
- **d)** D C F E

W7-M10 [what] A queue on a singly linked list with `head` and `tail` references should put the front of the queue at:

- **a)** the tail
- **b)** the middle node
- **c)** either end — it makes no difference
- **d)** the head

W7-M11 [why] Why not dequeue at the **tail** of a singly linked list?

- **a)** Removing the last node needs the node before it, which takes an $O(n)$ walk
- **b)** The tail node has no value
- **c)** It would make the queue LIFO
- **d)** Python forbids it

W7-M12 [how] A queue built from two stacks (`inbox` and `outbox`, pouring only when `outbox` is empty): `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`. How many items are now in `outbox`?

- **a)** 0
- **b)** 1
- **c)** 2
- **d)** 3

W7-M13 [why] The cost of dequeue on that two-stack queue is:

- **a)** $O(n)$ on average
- **b)** $O(\log n)$
- **c)** $O(1)$ in the worst case
- **d)** amortised $O(1)$ — each item is poured from `inbox` to `outbox` at most once

W7-M14 [what] Which of these needs a **queue**?

- **a)** checking balanced brackets
- **b)** undo in an editor
- **c)** breadth-first search of a graph
- **d)** evaluating a postfix expression

W7-M15 [how] n enqueues followed by n dequeues on a `SlowQueue` cost in total:

- **a)** $\Theta(n^2)$
- **b)** $\Theta(n)$
- **c)** $\Theta(n \log n)$
- **d)** $\Theta(1)$

W7-M16 [how] In Python, `(0 - 1) % 8` is:

- **a)** -1
- **b)** 7
- **c)** 0

- **d)** an error

W7-M17 [what] A **deque** is a structure that:

- **a)** serves the most urgent item first
- **b)** can only be used as a stack
- **c)** removes items in sorted order
- **d)** adds and removes at both ends

W7-M18 [why] Measured, a ring-buffer dequeue and a linked-queue dequeue are both flat lines as n grows, but the ring is about five times slower. The reason:

- **a)** a constant factor: the % and the bounds-checked Array access on every operation
- **b)** the ring is really $O(\log n)$
- **c)** the ring copies the queue on every dequeue
- **d)** the linked queue is $O(1)$ and the ring is $O(n)$

W7-M19 [how] Five people A, B, C, D, E stand in a circle; counting from A, every **second** person leaves. Who is left last?

- **a)** A
- **b)** E
- **c)** C
- **d)** D

W7-M20 [how] An array queue with a head index but **no** wrap-around, 8 slots: 8 enqueues, then 5 dequeues. How many more enqueues fit without growing the block?

- **a)** 5
- **b)** 0
- **c)** 3
- **d)** 8

W7-M21 [why] A CircularQueue that doubles its capacity when full has an enqueue cost of:

- **a)** $O(n)$ every time
- **b)** $O(\log n)$
- **c)** $O(1)$ in the worst case
- **d)** amortised $O(1)$

W7-M22 [what] A **priority queue** (Week 12) differs from a queue because it dequeues:

- **a)** the most urgent item, not the oldest
 - **b)** the newest item
 - **c)** a random item
 - **d)** from both ends
-

Part B — Short answer and essay

W7-E1 [why] (4 marks) State the Queue ADT with the cost of each operation. Explain why an array-based queue that shifts is $O(n)$ per dequeue, and why swapping which end is the front does not help. How does a ring buffer fix it?

W7-E2 [how] (4 marks) In a ring buffer, `head == tail` can mean two different things. Explain which two, with an example of each, and describe two ways of telling them apart. Which one does `dsa/queue.py` use?

W7-E3 [why] (4 marks) A full `CircularQueue` must grow. Explain, with an example, why copying the block slot by slot gives a wrong queue once the data has wrapped, give the correct copy, and explain why doubling keeps enqueue amortised $O(1)$.

W7-E4 [why] (3 marks) Describe a queue built from two stacks. Show that it is amortised $O(1)$ per operation, and explain why pouring must happen only when the outbox is empty.

W7-E5 [why] (3 marks) Compare the ring buffer and the linked queue (with head and tail): cost per operation, memory, and when you would choose each. Where does Python's `collections.deque` fit?

Part C — Trace the code

W7-T1 [how] What is printed? Then say which slot of the block holds each value at the end.

```
q = CircularQueue(capacity=4)
out = []
for v in (1, 2, 3):
    q.enqueue(v)
out.append(q.dequeue())
q.enqueue(4)
q.enqueue(5)
out.append(q.dequeue())
q.enqueue(6)
out.append(q.is_full())
out.append(list(q))
print(out)
```

W7-T2 [how] A two-stack queue (W7-C1) runs: `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`, `enqueue(4)`, `dequeue()`, `dequeue()`, `enqueue(5)`, `dequeue()`, `dequeue()`. Show both stacks (bottom to top) after every operation, and what each dequeue returns.

W7-T3 [how] Six people A–F stand in a circle; counting from A, every **third** person leaves (W7-C2). Show the queue, front to back, after each person leaves. In what order do they leave, and who is last?

W7-T4 [how] `binary_numbers(6)` (W7-C5) starts with the queue `["1"]`. Show the queue after each dequeue (and its two enqueues). What is returned?

W7-T5 [why] What is printed? Why are the two halves of the output in different orders?

```
def mystery(values):
    q, s = CircularQueue(len(values)), Stack()
    for v in values:
        q.enqueue(v)
    while len(q) > 0:
        v = q.dequeue()
        if v % 2 == 0:
            s.push(v)
        else:
            print(v, end=" ")
    while not s.is_empty():
        print(s.pop(), end=" ")

mystery([1, 2, 3, 4, 5, 6])
```

Part D — Ring state

W7-S1 [how] Draw the block, head and size after each operation on `q = CircularQueue(capacity=5)`. What is `list(q)` at the end, and is the queue full?

```
enqueue 10, enqueue 20, enqueue 30, enqueue 40, dequeue, dequeue,
enqueue 50, enqueue 60, enqueue 70, dequeue
```

W7-S2 [why] A student computes the tail as `self._head + self._size`, without `% self._capacity`. Draw what happens in `test_circular_queue_wraps_around`: capacity 3, enqueue 1, 2, 3, dequeue, enqueue 4. Why did the tests of earlier operations not catch it?

W7-S3 [how] A deque on a ring buffer, like `CircularQueue`, keeps head and size. `add_front` first steps head back one slot, `(head - 1) % capacity`, then writes there; `add_back` writes at `(head + size) % capacity`; `remove_back` takes the slot `(head + size - 1) % capacity`. Starting from an empty deque of capacity 4, draw the block, head and size after each operation, and the deque front to back:

```
add_back 1, add_back 2, add_front 0, add_front 9, remove_back,
add_back 5, remove_front, add_front 7, add_back 8
```

The last operation finds the deque full: it doubles the capacity first.

Part E — Complexity analysis

W7-K1 [how] Give Θ and justify:

```
q = SlowQueue()
for i in range(n):
    q.enqueue(i)
while len(q) > 0:
    q.dequeue()
```

What is it with a `CircularQueue(n)` instead?

W7-K2 [how] Give Θ and justify:

```
q = CircularQueue(n)
for i in range(n):
    q.enqueue(i)
print(sum(q))                # sum iterates over the whole queue
```

W7-K3 [why] A student's two-stack queue pours inbox into outbox on **every** dequeue, and pours it back afterwards. Give the cost of n enqueues followed by n dequeues, and compare with the correct version.

Part F — Find and fix the bug

W7-B1 [how] `SlowQueue`:

```
def dequeue(self):
    if len(self._items) == 0:
        raise IndexError("dequeue from empty queue")
    return self._items.pop()
```

W7-B2 [why] `CircularQueue` — this passes `test_circular_queue_wraps_around`:

```
def dequeue(self):
    if self._size == 0:
        raise IndexError("dequeue from empty queue")
    value = self._block[self._head]
    self._block[self._head] = None
    self._head += 1
    self._size -= 1
    return value
```

W7-B3 [how] `CircularQueue`:

```
def enqueue(self, value):
    tail = (self._head + self._size) % self._capacity
    if tail == self._head:
        raise IndexError("queue is full")
    self._block[tail] = value
    self._size += 1
```

W7-B4 [why] A linked queue with head and tail:

```
def dequeue(self):
    if self.head is None:
        raise IndexError("dequeue from empty queue")
    value = self.head.value
    self.head = self.head.next
    return value
```

Part G — Write the code

In `practice/week07.py`; check with `pytest tests/test_practice_week07.py -v`. Use your own `CircularQueue`, `Stack` or the course `Array` as working storage.

W7-C1 [why] `StackQueue` — a queue from two stacks. Why is it amortised $O(1)$?

W7-C2 [how] `josephus(names, k)` — the order in which people leave the circle.

W7-C3 [why] `moving_averages(values, k)` — the average of every window of k values, with a `CircularQueue` as the window. Why is it $O(n)$ and not $O(nk)$?

W7-C4 [how] `reverse_first_k(values, k)` — with one queue and one stack.

W7-C5 [how] `binary_numbers(n)` — the binary strings 1 to n , generated breadth-first.